

Probabilistic Analysis and Quick Sort Notes

Introduction

This technical note summarizes the tools and logic used in probabilistic analysis.

Type of probabilistic analysis

There are four main types of probabilistic analysis.

- Random if
 - With Recursion
 - Without recursion
- Random for
 - single random loop
 - Random loop inside an outer loop
- Random-sized recursive call (i.e. Number line problem)
- Loop that randomly terminates

1. Random if

Method: *Find the probabilities of both conditions and their corresponding $T(n)$, then use the formula.*

1.1 Random if with recursion

```

1. k = Random(n)
2. if Boolean condition on k then
3.     Make recursive call
4. else
5.     Make recursive call
6. end if

```

For example

```

function Function1(A[], n) {
    k = random(n)
    if (k mod 4 = 1) {
        x = x + Function1(A[], n/2)
    } else {
        x = x + Function1(A[], n/3)
    }
}

```

We can use the following formula:

$$ET(n) = \Pr(A) * E(T(n) | A) + \Pr(\text{Not } A) * E(T(n) | \text{Not } A)$$

From the formula, we can easily see that

$$ET(n) = \frac{1}{4} * T\left(\frac{n}{2}\right) + \left(1 - \frac{1}{4}\right) * T\left(\frac{n}{3}\right)$$

1.2 Random if without recursion

```

1. k = Random(n)
2. if Boolean condition on k then
3.     Foo()
4. else
5.     Bar()
6. end if

```

For example

```

function Function2(A[], n) {
    k = random(n)
    int x = 0;
    if (k mod 4 = 1) {
        for (i = 1 to n) {
            x = x + A[i];
        }
    } else {
        int i = 1;
        while (i <= n) {
            x = x + A[i];
            i = 2 * i;
        }
    }
}

```

We can use the following formula (Same as in Section 1.1):

$$ET(n) = \Pr(A) * E(T(n) \mid A) + \Pr(\text{Not } A) * E(T(n) \mid \neg A)$$

OR

$$ET(n) = \Pr(A) * E(T(n) \mid A) + \Pr(\text{Not } A) * E(T(n) \mid \text{NOT } A)$$

OR

$$ET(n) = \Pr(\text{Foo}) * E(T(n) \mid \text{Foo}) + \Pr(\text{Bar}) * E(T(n) \mid \text{Bar})$$

Since

$$\Pr(A) = \frac{1}{4}, E(T(n) \mid \text{Foo}) = \Theta(n)$$

and

$$\Pr(\neg A) = \frac{3}{4}, E(T(n) \mid Bar) = \Theta(\log n)$$

From the formula, we can easily get that

$$ET(n) = \frac{1}{4} * \Theta(n) + \frac{3}{4} * \Theta(\log n)$$

2. Random for

Method: *First analyze the running time for a fixed random value of k , and then incorporate the result into the outer loop if there is one.*

2.1 single random loop

1. $k = \text{Random}(n)$
2. Loop that runs $f(k)$ times

For example

```
function Function3(A[], n) {
    x = 0;
    k = Random(n);
    for (i = 1 to k){
        for(j = 1 to i * i) {
            x = x + A[ij mod n];
        }
    }

    return x;
}
```

We can use the formula:

$$ET(n) = \sum_{q=1}^n \Pr(k = q)t(k = q)$$

We should first find $t(k=q)$, which is the running time of the loop controlled by k .

$$t(k = q) = \left(\sum_{i=1}^q i^2 C \right)$$

Since k is uniformly random,

$$\Pr(k = q) = \frac{1}{n}$$

Therefore,

$$ET(n) = \sum_{q=1}^n \frac{1}{n} \sum_{i=1}^q i^2 C$$

2.2 Random loop inside an outer loop

```

1.  loop
2.      k = Random(i)
3.      Loop that runs f(i) times
4.  end loop

```

For example

```

function Function4(A[], n) {
    x = 0;
    for (i = 1 to n) {
        k = Random(i);
        for (j = 1 to k * k){
            x = x + A[(i*j mod n)]
        }
    }

    return x;
}

```

For each iteration of the outer loop, we first analyze the time complexity of the inner loop.

We can use the formula:

$$ET_i(n) = \sum_{q=1}^i \Pr(k = q) t(k = q)$$

Since k is uniformly random from 1 to i ,

$$\Pr(k = q) = \frac{1}{i}$$

and because the inner loop runs k^2 times,

$$t(k = q) = q^2 C$$

Therefore,

$$\begin{aligned}
 ET_i(n) &= \sum_{q=1}^i \frac{1}{i} \Theta(q^2) \\
 &= \frac{1}{i} \Theta(i^3) = \Theta(i^2)
 \end{aligned}$$

Then, we can determine the overall time complexity.

$$ET(n) = \sum_{i=1}^n ET_i(n)$$

$$= \sum_{i=1}^n \Theta(i^2) = \Theta(n^3)$$

3. Random-sized recursive call (i.e. Number axis problem)

Method: *First identify the best and worst cases. Then use a number line to divide the possible values of k into ranges, and use the appropriate best- or worst-case bound in each range to determine the overall time complexity.*

1. Base case and non-recursive work
2. Do ()
3. $k = \text{Random}(n)$
4. Make recursive call of size $f(k)$.
4. return

For example

```
function Function5(A[], n) {
    if (n <= 22) {
        return 1;
    }
    x = 0;
    for (i = 1 to n/2) {
        for (j = 1 to n/4) {
            x = x + A[(ij mod n)];
        }
    }
    k = Random(n/5 - 1);
    x = x + Func5(A[], 5k);
    x = x + Func5(A[], n - 5k);

    return x;
}
```

We use the following steps:

1. Substitute the minimum and maximum possible values of k to identify the worst-case recursive-call sizes.
2. Find the midpoint where the two recursive calls have the same size.
3. Divide the possible values of k into several ranges using the critical points on the number line.
4. For each range, choose the value of k that gives the worst recursive-call sizes, then use these cases to construct the upper bound.
5. The process for finding the lower bound is similar, but we use the best case instead.

Solution:

At the two endpoints,

$$k = 1 \Rightarrow T(5) + T(n - 5)$$

and

$$k = \frac{n}{5} - 1 \Rightarrow T(n - 5) + T(5).$$

Both endpoints give the same unbalanced recursive-call sizes.

To find the midpoint, set the two recursive-call sizes equal:

$$5k = n - 5k$$

$$10k = n$$

$$k = \frac{n}{10}.$$

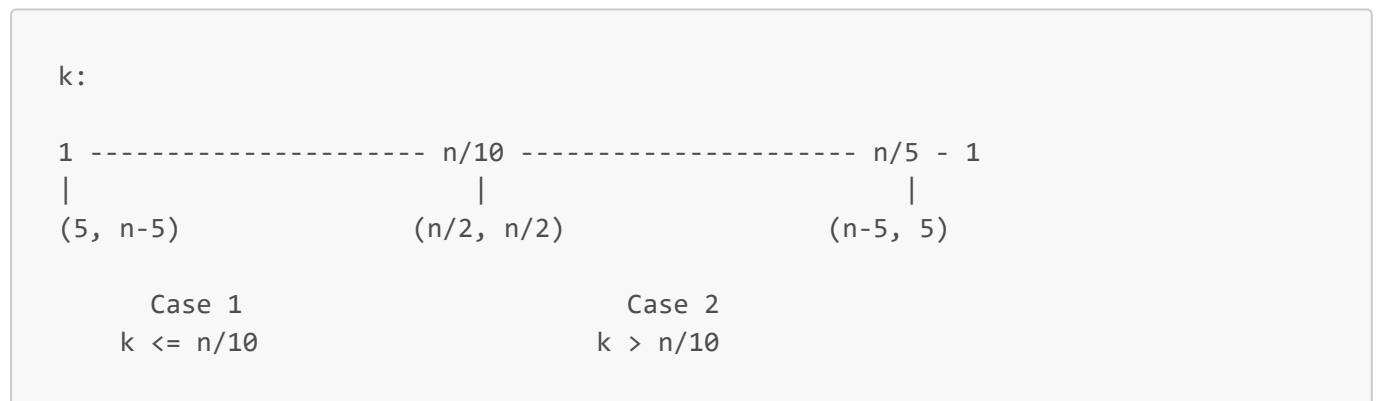
Therefore,

$$5k = n - 5k = \frac{n}{2},$$

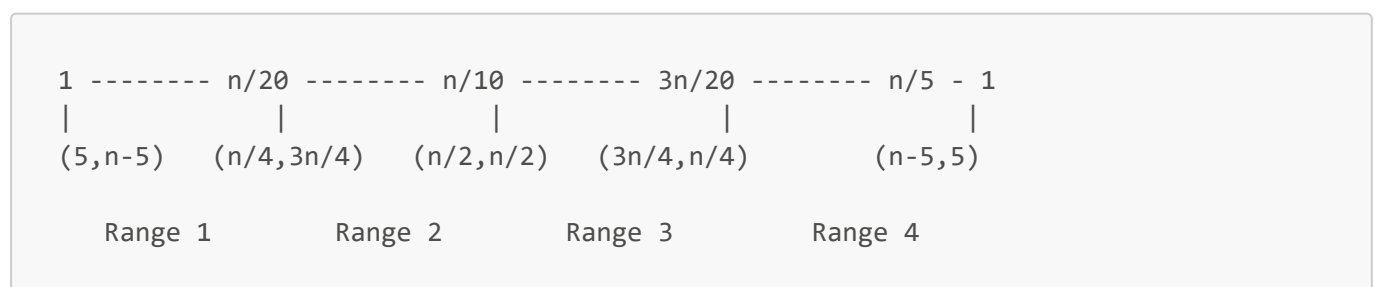
so the recursive cost at the midpoint is

$$2T(n/2).$$

Then we draw the number line:



Next, divide the possible values of k into four ranges:



For the upper bound, choose the worst recursive-call sizes in each range.

Therefore,

$$ET(n) \leq \Theta(n^2) + \frac{1}{4} [ET(5) + ET(n-5)] + \frac{1}{4} [ET(n/4) + ET(3n/4)] \\ + \frac{1}{4} [ET(3n/4) + ET(n/4)] + \frac{1}{4} [ET(n-5) + ET(5)].$$

Combining identical terms gives

$$ET(n) \leq \Theta(n^2) + \frac{1}{2} [ET(5) + ET(n-5)] + \frac{1}{2} [ET(n/4) + ET(3n/4)].$$

Since $ET(5) = \Theta(1)$, it can be absorbed into the other asymptotic terms.

For the upper bound, we can also replace $ET(n-5)$ by $ET(n)$ because

$$n-5 < n$$

and therefore

$$ET(n-5) \leq ET(n).$$

Thus,

$$ET(n) \leq \Theta(n^2) + \frac{1}{2} ET(n) + \frac{1}{2} ET(n/4) + \frac{1}{2} ET(3n/4).$$

Move $\frac{1}{2} ET(n)$ to the left-hand side:

$$ET(n) - \frac{1}{2} ET(n) \leq \Theta(n^2) + \frac{1}{2} ET(n/4) + \frac{1}{2} ET(3n/4).$$

Therefore,

$$\frac{1}{2} ET(n) \leq \Theta(n^2) + \frac{1}{2} ET(n/4) + \frac{1}{2} ET(3n/4).$$

Multiplying both sides by 2 gives

$$ET(n) \leq ET(n/4) + ET(3n/4) + \Theta(n^2).$$

Therefore,

$$ET(n) = O(n^2).$$

For the Lower Bound, the expected running time cannot be better than the best possible recursive split.

The best split occurs at

$$k = \frac{n}{10},$$

which gives two recursive calls of size $n/2$.

Therefore, the best-case recurrence is

$$T(n) = 2T(n/2) + \Theta(n^2).$$

This recurrence gives

$$T(n) = \Theta(n^2).$$

Since the expected running time cannot be better than the best-case running time,

$$ET(n) = \Omega(n^2).$$

Therefore,

$$ET(n) = \Theta(n^2)$$

4. Loop that randomly terminates

```

1. Do some work
2. repeat
3.     k = Random(n)
4.     Do some work
5. until Boolean condition on k is true

```

For this type of problem, the loop terminates only when a random condition is satisfied.

We let X be the number of times the loop repeats before termination.

For example:

```

function Function6(A[], n) {
    x = 0;

    repeat {
        k = Random(n);

        for (i = 1 to sqrt(n)) {
            x = x + A[i];
        }

    } until (k == 42);

    return x;
}

```

The loop terminates when

$$k = 42.$$

Since k is uniformly random from 1 to n ,

$$\Pr(k = 42) = \frac{1}{n}$$

and

$$\Pr(k \neq 42) = \frac{n-1}{n}.$$

The best case occurs when

$$k = 42$$

on the first random choice.

Therefore,

$$T_{\text{best}}(n) = \Theta(\sqrt{n}).$$

In the worst case, k may never equal 42, so the loop has no finite deterministic worst-case bound.

For the expected case, let X be the number of times the loop repeats before termination.

The running time can be written as

$$T(n) = c\sqrt{n} + X \cdot c\sqrt{n}.$$

Therefore,

$$ET(n) = c\sqrt{n} + E(X) \cdot c\sqrt{n}.$$

To find $E(X)$, use

$$E(X) = \sum_{i=1}^{\infty} \Pr(X \geq i).$$

For the loop to repeat at least i times, the terminating condition must fail repeatedly. Thus,

$$\Pr(X \geq i) = \left(\frac{n-1}{n}\right)^i.$$

Therefore,

$$E(X) = \sum_{i=1}^{\infty} \left(\frac{n-1}{n}\right)^i.$$

Using the geometric-series formula,

$$\sum_{i=1}^{\infty} r^i = \frac{r}{1-r},$$

we get

$$E(X) = \frac{\frac{n-1}{n}}{1 - \frac{n-1}{n}}.$$

Since

$$1 - \frac{n-1}{n} = \frac{1}{n},$$

we have

$$E(X) = \frac{n-1}{n} \cdot n = n-1.$$

Thus,

$$ET(n) = c\sqrt{n} + (n-1)c\sqrt{n}.$$

Therefore,

$$ET(n) = cn\sqrt{n}.$$

So,

$$ET(n) = \Theta(n^{3/2})$$

Reference List

1. "CSE2331_Types_of_Probabilistic_Problems" Created by Professor Painter.
2. "Quick Sort Worksheet AU26" Created by Professor Painter.
3. "CSE2331_QuickSort_and_Probabilistic_Analysis_Homework" Created by Professor Painter.